Assessment Tool 2 — Internet Programming (CSA43)

CO-2: CSS Layout, Styling & JavaScript Interactivity — Full Solutions

Question 1: Responsive Navigation Bar Design

Scenario: An e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

(a) Key components of an e-commerce navigation bar

- **Logo/brand** — links back to the home page.
- **Primary links** — Home, Categories, Deals, About, Contact.
- **Search bar** — prominent, since product discovery is central to e-commerce.
- **Cart icon** — usually with an item-count badge.
- **Account/login** — sign-in or profile menu.
- **Category/mega-menu dropdown** — for browsing product categories.
- **Hamburger toggle** — appears on smaller screens to collapse the above into a menu.

(b) Responsive navigation bar using CSS

```
<header class="navbar">
  <a href="#" class="brand">ShopEase</a>
  <button class="menu-toggle" aria-label="Toggle menu" aria-
expanded="false">☰</button>
  <nav class="nav-links">
    <a href="#">Home</a>
    <a href="#">Categories</a>
    <a href="#">Deals</a>
    <a href="#">Cart (3)</a>
    <a href="#">Account</a>
  </nav>
</header>
```

```
* { box-sizing: border-box; margin: 0; padding: 0; }

.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0.8rem 1.5rem;
  background: #1f2937;
  color: #fff;
}
.brand { color: #fff; font-weight: 700; text-decoration: none; font-size:
```

```
1.2rem; }

.nav-links {
  display: flex;
  gap: 1.5rem;
}
.nav-links a { color: #fff; text-decoration: none; }

.menu-toggle {
  display: none;
  background: none;
  border: none;
  color: #fff;
  font-size: 1.5rem;
  cursor: pointer;
}
```

Flexbox (`display: flex; justify-content: space-between;`) keeps the brand, links, and icons aligned in a single row on desktop.

(c) Adapting to smaller screens (hamburger menu)

```
@media (max-width: 768px) {
  .menu-toggle { display: block; }

  .nav-links {
    display: none;
    flex-direction: column;
    position: absolute;
    top: 56px;
    left: 0;
    right: 0;
    background: #1f2937;
    padding: 1rem;
  }

  .nav-links.open { display: flex; }
}

const toggle = document.querySelector('.menu-toggle');
const links = document.querySelector('.nav-links');

toggle.addEventListener('click', () => {
  const isOpen = links.classList.toggle('open');
  toggle.setAttribute('aria-expanded', isOpen);
});
```

Below 768px, the horizontal link list is hidden and replaced by a hamburger icon; clicking it toggles a vertical dropdown menu using a simple class toggle — no page reload or heavy JS framework needed.

(d) Usability evaluation and improvements

Evaluation: The layout keeps all primary actions reachable on desktop and collapses gracefully on mobile, preserving screen space for product content. Using `aria-expanded` gives basic accessibility support for the toggle state.

Improvements: 1. Add a **close-on-outside-click** and **Escape key** handler so the mobile menu doesn't stay open unexpectedly. 2. Use a **slide-in animation** (`transition: transform`) rather than an abrupt show/hide for a smoother feel. 3. Make the **search bar always visible** (even collapsed to an icon) since search is a primary e-commerce action, not just a nav link. 4. Add **focus trapping** inside the open mobile menu for keyboard/screen-reader users. 5. Persist an **active-link indicator** (e.g., underline) so users know which section they're on.

Question 2: Product Hover Effect

Scenario: An e-commerce website wants to enhance user interaction through visual effects on product items.

(a) Suitable CSS properties for hover effects

- `:hover` pseudo-class — triggers style changes on mouse-over.
- `transform` (`scale()`, `translateY()`) — moves/scales the card without affecting layout flow.
- `box-shadow` — adds depth/elevation on hover.
- `opacity` — used for overlay reveal effects (e.g., "Add to Cart" button fading in).
- `filter: brightness()/filter: blur()` — subtle image emphasis effects.
- `transition` — animates the property changes smoothly rather than snapping instantly.

(b) Suggested transition/animation

```css
.product-card {
  transition: transform 0.25s ease, box-shadow 0.25s ease;
}
```

A **`transition`** (rather than `@keyframes` animation) is the right tool here because the effect is a simple two-state change (normal → hovered) driven by user interaction, not a continuous or looping animation. `ease` timing feels natural for UI micro-interactions, and 200–300ms is fast enough to feel responsive without being jarring.

(c) Hover effect for a product card

```html
<div class="product-card">
  <img src="product.jpg" alt="Product name" />
  <div class="overlay">
    <button>Add to Cart</button>
```

```
  </div>
  <h4>Product Name</h4>
  <p>$29.99</p>
</div>

.product-card {
  position: relative;
  overflow: hidden;
  border-radius: 10px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
  transition: transform 0.25s ease, box-shadow 0.25s ease;
}
.product-card:hover {
  transform: translateY(-6px) scale(1.02);
  box-shadow: 0 12px 24px rgba(0,0,0,0.15);
}

.product-card img {
  width: 100%;
  display: block;
  transition: transform 0.4s ease;
}
.product-card:hover img { transform: scale(1.08); }

.overlay {
  position: absolute;
  inset: auto 0 0 0;
  background: rgba(0,0,0,0.55);
  display: flex;
  justify-content: center;
  padding: 0.6rem;
  opacity: 0;
  transform: translateY(100%);
  transition: opacity 0.25s ease, transform 0.25s ease;
}
.product-card:hover .overlay {
  opacity: 1;
  transform: translateY(0);
}
```

On hover, the card lifts (`translateY` + `scale`), the shadow deepens for a sense of elevation, the product image zooms slightly, and an “Add to Cart” overlay slides up from the bottom.

(d) Performance and UX evaluation

Performance considerations: - Only **`transform`** and **`opacity`** are animated — both run on the GPU compositor and don’t trigger layout reflow, keeping the effect smooth even on lower-end devices. - Avoid animating `width`, `height`, `top`/`left`, or `box-shadow` blur radius on large lists, as these force repaints; here `box-shadow` is a single low-cost change but could be swapped for a pseudo-element if profiling shows jank on long product grids. - `will-`

`change: transform` can be added for cards known to animate frequently, but should be used sparingly (it consumes GPU memory).

UX considerations: 1. Hover effects don't work on **touch devices** — ensure the "Add to Cart" action is also reachable via tap/always-visible on mobile, not hover-only. 2. Keep motion **subtle** (a few pixels/percent) so it doesn't feel distracting when scrolling past many cards. 3. Respect `prefers-reduced-motion` media query to disable transforms for users who opt out of animations. 4. Ensure hover states have a **keyboard-focus equivalent** (`:focus-visible`) so keyboard users get the same affordance.

Question 3: Layout Selection (Flexbox vs Grid) for a Dashboard

Scenario: A dashboard layout needs to be designed for a web application.

(a) Layout requirements of a dashboard

- A **fixed sidebar** (navigation) and a **flexible main content** area.
- A **top header/toolbar** row spanning the full width.
- A **grid of widgets/cards** (charts, KPIs, tables) that must align in both rows and columns.
- Consistent **gutters/spacing** between widgets.
- Widgets of **varying sizes** (some spanning 2 columns, some 1) that must still align to a shared grid.
- **Responsiveness** — sidebar collapses and widgets reflow to fewer columns on smaller screens.

(b) Flexbox vs Grid comparison

Aspect	Flexbox	Grid
Dimension	One-dimensional (row *or* column at a time)	Two-dimensional (rows *and* columns together)
Best for	Aligning items within a single row/column, e.g. a toolbar or nav	Overall page skeleton and widget grids that must align across rows and columns
Content-driven sizing	Sizes based on content by default	Can define explicit track sizes independent of content
Item spanning	No native concept of "spanning" multiple rows/columns	`grid-column/grid-row span` lets widgets span multiple cells cleanly
Alignment control	Strong for distributing items along one axis (`justify-content`, `align-items`)	Strong for placing items precisely in a 2D coordinate grid
Typical dashboard	Toolbar, sidebar internal nav items, card internals	Overall dashboard skeleton (sidebar + header + widget area) and the

Aspect	Flexbox	Grid
use		widget grid itself

(c) Chosen technique and justification

CSS Grid is the most suitable technique for the overall dashboard skeleton and widget area, because a dashboard is fundamentally a **two-dimensional** layout problem: widgets must align both horizontally (columns of equal width) and vertically (rows), and some widgets need to **span multiple columns/rows** (e.g., a large chart spanning 2 columns next to two stacked KPI cards). Grid's `grid-template-columns`, `grid-template-areas`, and `span` give this control directly, without the nested wrapper divs Flexbox would require to fake a 2D grid.

Flexbox is still used *within* the Grid — e.g., to align icon + text inside the header, or to center content inside an individual widget card — since those are one-dimensional alignment problems. This "Grid for skeleton, Flexbox for components" pairing is standard practice.

```
.dashboard {
  display: grid;
  grid-template-columns: 220px 1fr;
  grid-template-rows: 60px 1fr;
  grid-template-areas:
    "sidebar header"
    "sidebar main";
  height: 100vh;
}
.sidebar { grid-area: sidebar; }
.header  { grid-area: header; display: flex; align-items: center; padding: 0
1rem; }
.main    { grid-area: main; padding: 1rem; }

.widget-grid {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 1rem;
}
.widget--wide { grid-column: span 2; }
.widget--tall { grid-row: span 2; }
```

(d) Scalability and responsiveness evaluation

- **Scalability:** Adding a new widget is just a new grid item; `grid-template-columns: repeat(4, 1fr)` scales automatically to accommodate more items without restructuring existing CSS, and `span` values let designers resize individual widgets without touching the surrounding grid.
- **Responsiveness:** A single media query can redefine the whole skeleton — e.g., collapsing `grid-template-columns: 220px 1fr` to `1fr` and switching `grid-`

`template-areas` to stack the sidebar above main content on mobile — which is far simpler than re-plumbing multiple nested Flexbox containers.

```
@media (max-width: 768px) {
  .dashboard {
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "main";
  }
  .sidebar { display: none; } /* replaced by a toggleable drawer */
  .widget-grid { grid-template-columns: repeat(2, 1fr); }
}
@media (max-width: 480px) {
  .widget-grid { grid-template-columns: 1fr; }
}
```

This confirms Grid scales cleanly from a 4-column widget layout down to 2 and then 1 column, while the dashboard skeleton itself collapses from sidebar+main to a stacked single column.

Question 4: Mobile-First Blog Layout

Scenario: A blog page must be designed following a mobile-first approach.

(a) Key sections of a blog layout

- **Header** — site title/logo, optional nav toggle.
- **Post title, meta info** — author, date, read time.
- **Featured image**.
- **Post body/content** — text, images, blockquotes, code blocks.
- **Tags/categories**.
- **Author bio / share buttons**.
- **Comments section**.
- **Related posts**.
- **Footer**.

(b) Mobile-first CSS design

Mobile-first means writing the **base (unprefixed) CSS for the smallest screen first**, then layering `min-width` media queries to enhance the layout for larger screens — rather than starting from desktop and overriding downward.

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font: 1rem/1.6 system-ui, sans-serif; color: #1f2937; }
```

```css
/* Base (mobile) styles: single column, generous line-height for readability
*/
.post {
  padding: 1rem;
  max-width: 100%;
}
.post img { width: 100%; height: auto; border-radius: 8px; }
.post h1 { font-size: 1.5rem; margin-bottom: 0.5rem; }
.post-meta { font-size: 0.85rem; color: #6b7280; margin-bottom: 1rem; }

.related-posts {
  display: grid;
  grid-template-columns: 1fr; /* one column by default on mobile */
  gap: 1rem;
}
```

(c) Scaling up with media queries

```css
/* Tablet and up */
@media (min-width: 600px) {
  .post { padding: 1.5rem 2rem; }
  .post h1 { font-size: 2rem; }
  .related-posts { grid-template-columns: repeat(2, 1fr); }
}

/* Desktop and up */
@media (min-width: 992px) {
  .post {
    max-width: 720px;   /* comfortable reading line-length */
    margin: 0 auto;
    padding: 2rem 0;
  }
  .related-posts { grid-template-columns: repeat(3, 1fr); }
}
```

Using `min-width` queries (rather than `max-width`) is the hallmark of mobile-first CSS: styles are additive as the screen grows, instead of starting with a full desktop layout and stripping it down.

(d) Accessibility and readability evaluation

Evaluation: - Base font-size of `1rem` (16px) and `line-height: 1.6` meet common readability guidelines for body text. - Constraining desktop reading width to ~`720px` (≈60–75 characters per line) improves reading comprehension, a widely recommended typographic practice. - Single-column mobile layout matches natural top-to-bottom scroll reading, with no horizontal scrolling.

Improvements: 1. Ensure **color contrast** (body text vs. background) meets WCAG AA (4.5:1) — verify with a contrast checker. 2. Use **semantic HTML** (`<article>`, `<header>`, `<time datetime="">`, `<nav>`) so screen readers can navigate the structure. 3. Add **`alt` text**

for the featured image and all inline images. 4. Support **`prefers-color-scheme: dark`** for readers who prefer dark mode, especially for long-form reading. 5. Use **relative units (`rem`/`em`)** throughout (already done above) so text scales correctly if the user increases browser font size.

Question 5: JavaScript Event Handling for Dynamic Menu

Scenario: A website includes a dynamic menu that responds to user interactions.

(a) Appropriate JavaScript events for menu interaction

- **`click`** — primary toggle action for opening/closing the menu (works for both mouse and touch).
- **`mouseenter` / `mouseleave`** — for desktop hover-triggered dropdown submenus.
- **`keydown`** — to support keyboard navigation (Enter/Space to open, Escape to close, Arrow keys to move between items).
- **`focus` / `blur`** (or `focusin`/`focusout`) — to open/close submenus for keyboard users and detect when focus leaves the menu.
- **`document` click listener** — to detect clicks *outside* the menu and close it.
- **`resize`** — to reset menu state if the viewport crosses a breakpoint while the menu is open.

(b) Event handling logic for menu toggle

```
<button id="menuBtn" aria-haspopup="true" aria-expanded="false">Menu</button>
<ul id="menuList" class="dropdown" hidden>
  <li><a href="#">Profile</a></li>
  <li><a href="#">Settings</a></li>
  <li><a href="#">Logout</a></li>
</ul>

const menuBtn = document.getElementById('menuBtn');
const menuList = document.getElementById('menuList');

function openMenu() {
  menuList.hidden = false;
  menuBtn.setAttribute('aria-expanded', 'true');
}
function closeMenu() {
  menuList.hidden = true;
  menuBtn.setAttribute('aria-expanded', 'false');
}
function toggleMenu() {
  menuList.hidden ? openMenu() : closeMenu();
}
```

The logic is centralized into `openMenu`/`closeMenu`/`toggleMenu` functions so every event handler (click, keyboard, outside-click) reuses the same source of truth for state, avoiding inconsistent open/closed states across triggers.

(c) Implementing click/hover interaction

```
// Click to toggle (mobile & desktop)
menuBtn.addEventListener('click', (e) => {
  e.stopPropagation();
  toggleMenu();
});

// Close on outside click
document.addEventListener('click', (e) => {
  if (!menuList.hidden && !menuList.contains(e.target) && e.target !==
menuBtn) {
    closeMenu();
  }
});

// Keyboard support
menuBtn.addEventListener('keydown', (e) => {
  if (e.key === 'Enter' || e.key === ' ') {
    e.preventDefault();
    toggleMenu();
  }
});
document.addEventListener('keydown', (e) => {
  if (e.key === 'Escape') closeMenu();
});

// Optional: hover for desktop pointer devices only
if (window.matchMedia('(hover: hover)').matches) {
  const wrapper = menuBtn.parentElement;
  wrapper.addEventListener('mouseenter', openMenu);
  wrapper.addEventListener('mouseleave', closeMenu);
}
```

`e.stopPropagation()` on the button's click prevents the same click from immediately bubbling to the `document` listener and closing the menu it just opened. The `(hover: hover)` media query check avoids attaching hover handlers on touch devices, where hover doesn't have a reliable meaning.

(d) Performance evaluation and improvements

Evaluation: Attaching one `document`-level click/keydown listener (rather than one per menu item) keeps the number of bound listeners low and scales fine even with many menus on a page. Using `hidden` (vs. re-rendering DOM nodes) is cheap since it only toggles a CSS-level display change.

Improvements: 1. **Debounce/throttle** the `resize` listener if it's used to reset menu state, since `resize` fires very frequently. 2. Use **event delegation** for menu item clicks if the menu has many items, attaching one listener on the `<ul>` instead of one per `<li>`. 3. Add a **CSS transition** on open/close (instead of an abrupt `hidden` toggle) for smoother UX, driven by adding/removing a class rather than inline styles. 4. Ensure **focus management**: move focus to the first menu item on open, and return focus to the trigger button on close, for accessibility. 5. Clean up listeners (`removeEventListener`) if the menu component is dynamically destroyed, to avoid memory leaks in single-page applications.

Code of Conduct

I **MANDLA VISWATEJA** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____